

Hugging Face Flashcard Generator

Practical Lab Report — AI & ML

Student Name	Ayushmaan Shukla
Course	B.Tech in AI & ML
University	Shri Ramswaroop Memorial University (SRMU)
Roll No.	202510101240164

1. Objective

To build a Python-based flashcard generator using Hugging Face Inference Providers, generate five study flashcards for a selected topic, and compare two text-generation/chat models.

2. Tools and Setup

Component	Details
Language	Python
Library	huggingface_hub
Client	InferenceClient
Authentication	HF_TOKEN environment variable
Project folder	openai-api-lab/huggingface_flashcards
Virtual environment	Existing .venv in parent openai-api-lab folder
Test topic	Machine Learning

3. Model 1 — Llama

Model ID: meta-llama/Llama-3.1-8B-Instruct

Topic: Machine Learning **Response time:** 18.87 seconds

The model successfully generated five flashcards. The tone was direct and concise, with shorter answers and basic explanations. The requested five-card format was followed.

4. Model 2 — Qwen

Model ID: Qwen/Qwen3-4B-Instruct-2507

Topic: Machine Learning **Response time:** 3.13 seconds

The model successfully generated five flashcards. Its tone was more explanatory and friendly. Answers were longer and included more examples and explanations. The requested five-card format was followed.

Note: Qwen/Qwen2.5-7B-Instruct was initially tested but was not supported by an enabled Inference Provider, so Qwen/Qwen3-4B-Instruct-2507 was used for the final comparison.

5. Model Comparison

Observation	Model 1	Model 2
Model ID	meta-llama/Llama-3.1-8B-Instruct	Qwen/Qwen3-4B-Instruct-2507
Response time	18.87 sec	3.13 sec
Tone	Direct, concise	More explanatory, friendly
Answer length	Shorter	Longer
Format adherence	Followed 5-flashcard format	Followed 5-flashcard format
Other observations	Basic explanations	More detailed examples and explanations

6. Reflection Questions

1. What is a Hugging Face Model ID?

A Hugging Face Model ID is a unique name used to identify a specific AI model on Hugging Face.

2. What does InferenceClient do?

InferenceClient connects the Python program to Hugging Face inference services and sends requests to the selected AI model.

3. What changed after switching models?

The generated answers, tone, answer length, and response time changed.

4. What stayed the same?

The topic, prompt, number of flashcards, and overall Python program structure stayed the same.

5. How did tone and latency differ?

Model 1 took 18.87 seconds and gave more concise answers. Model 2 took 3.13 seconds and gave more detailed and explanatory answers.

7. Key Pattern

```

Authenticate
↓
Choose Model
↓
Send Input
↓
Receive Output

```

8. Security Note

The Hugging Face access token must never be hard-coded in the Python program, uploaded to GitHub, included in screenshots, shared in chats, or placed in shared files. It should remain securely stored as the HF_TOKEN environment variable.

9. Conclusion

The practical demonstrated how a Python application can authenticate with Hugging Face, select an inference model, send a prompt, and receive generated content. Switching the model changed response characteristics such as latency, tone, and answer length while the application workflow and requested output format remained consistent.